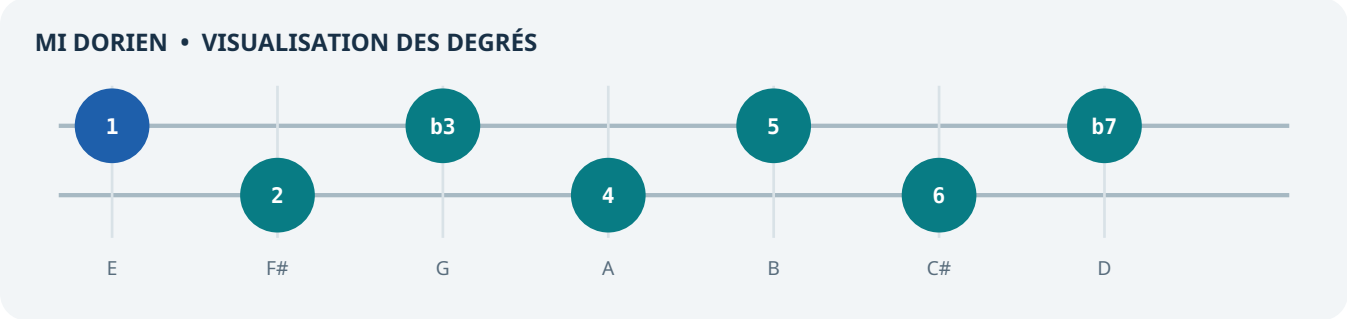
CAHIER DE CONSTRUCTION / CHAPITRE 04

Visualiser les intervalles sur le manche

De la distance chromatique au degré musical, puis à un affichage React interchangeable.



Stack React · TypeScript strict · Vite · pnpm	**Périmètre** Six cordes · cases 0–12 Mineur · majeur · dorien · phrygien
Résultat Affichage Notes / Intervalles sur les positions de la gamme	**Validation** Vérification manuelle réussie Build final non confirmé

Séance du 24 au 25 octobre 2026 · Version documentaire 1.0

Ce document constitue un chapitre autonome, prêt à être ajouté aux chapitres précédents. Les extraits reposent sur le code et les vérifications décrits au cours de la séance ; les détails des chapitres 1 à 3 non fournis ici ne sont pas reconstitués arbitrairement.

Objectif, prérequis et résultat

L'objectif de la séance 4 était de transformer une simple coloration des notes de gamme en une **lecture harmonique du manche** : pour chaque position, voir son degré par rapport à la tonique choisie (par exemple 1, 2, b3, 4, 5, 6, b7 en Mi dorien). L'utilisateur peut désormais alterner entre les noms des notes et les symboles d'intervalles.

État hérité des trois séances précédentes

- **Manche** : six cordes, cases 0 à 12, accordage standard E4–B3–G3–D3–A2–E2, position sélectionnée conservée par `useState`.
- **Logique musicale** : calcul de la hauteur MIDI et du nom des notes dans `src/music/notes.ts`.
- **Gammes** : `src/music/scales.ts` définit `CHROMATIC_SCALE`, `SCALES`, `ScaleName`, `modulo()`, `getTonicName()` et `isNoteInScale()`.
- **États existants** : `tonic` est une classe de hauteur 0 à 11 ; `scaleName` est typé `ScaleName`. Deux sélecteurs les pilotent.
- **Repères visuels** : notes de gamme vertes, toniques bleues et position sélectionnée orange. Le panneau `buildOutput()` fournit le nom MIDI et le contexte musical.

Ce qui a effectivement été ajouté

Couche	Incrément confirmé
Musique	Distance chromatique modulo 12 ; table des 12 degrés ; libellé d'intervalle.
TypeScript	Tuple as const ; union de littéraux ; narrowing dans le gestionnaire de changement.
React	Troisième état displayMode, select contrôlé, libellé calculé pendant le rendu.
Interface	Les positions de la gamme affichent leur nom ou leur degré ; une position hors gamme sélectionnée affiche aussi son libellé.

Décision de portée

Cette séance se limite à l'affichage. Aucun moteur audio, aucune tablature, aucun nouveau stockage et aucune architecture supplémentaire n'ont été introduits. Le mode et la tonique existants restent les sources de vérité.

> **À savoir pour reconstruire**
>
> Les interfaces de quelques fonctions déjà présentes (`getMidiNote()`, `getFullNoteName()`, `checkMidiRange()`) sont connues par leur utilisation dans la séance. Leurs corps et l'organisation complète des composants appartiennent aux chapitres antérieurs et doivent être repris de ce code source, ou réimplémentés séparément.

SÉQUENCE 01

Comprendre la distance et le degré

Une **distance chromatique** indique combien de demi-tons séparent deux hauteurs. Un **nom d'intervalle** fournit une interprétation musicale de cette distance à partir d'une tonique et, en théorie, de l'orthographe des notes.

Dans l'application, la tonique est une classe de hauteur (E = 4, F = 5, etc.). La hauteur MIDI de la case peut être dans n'importe quelle octave. Pour l'étiquetage des degrés, nous ignorons donc les octaves et ramenons la différence dans l'intervalle entier de 0 à 11.

```
distance = modulo(noteMidi - tonic, 12);
```

Exemple	Calcul brut	Classe d'intervalle
E4 (64) G4 (67)	67 - 64 = 3	3 demi-tons b3
E4 (64) G3 (55)	55 - 64 = -9	modulo(-9, 12) = 3 b3
E4 (64) E5 (76)	76 - 64 = 12	modulo(12, 12) = 0 1
E4 (64) D4 (62)	62 - 64 = -2	modulo(-2, 12) = 10 b7

Convention de notation retenue

Pour cette première version, une distance possède un libellé pratique et unique. Cela suffit pour reconnaître rapidement les degrés des quatre modes actuellement disponibles.

Demi-tons	Degré	Demi-tons	Degré	Demi-tons	Degré
0	1	4	3	8	b6
1	b2	5	4	9	6
2	2	6	#4	10	b7
3	b3	7	5	11	7

Limite musicale assumée

La distance chromatique **ne suffit pas à identifier toutes les orthographes d'intervalles** : E–F# (seconde majeure) et E–Gb (tierce diminuée) font tous deux 2 demi-tons. Notre table choisit 2 pour cette distance. De même, six demi-tons sont ici notés #4, pas b5.

Exemple de contrôle : Mi dorien

Les notes E, F#, G, A, B, C#, D doivent afficher respectivement **1, 2, b3, 4, 5, 6, b7**. C (Do naturel), en dehors de Mi dorien, correspond à **b6** lorsqu'elle est sélectionnée.

SÉQUENCE 02

Calculer la distance chromatique

Première fonction ajoutée dans `src/music/scales.ts` : calculer la distance ascendante modulo 12, en validant les hauteurs MIDI reçues.

```
export function getIntervalSemitones(
  tonicMidi: number,
  noteMidi: number
): number {
  checkMidiRange(noteMidi);
  checkMidiRange(tonicMidi);
  return modulo(noteMidi - tonicMidi, 12);
}
```

Pourquoi cette implémentation ?

- **Réutiliser** `modulo()` évite une deuxième logique de normalisation et protège contre les différences négatives.
- **Conserver une fonction pure en pratique** : à entrées égales, elle renvoie la même distance ; elle ne modifie aucun état React.
- **Valider les arguments** au moyen de `checkMidiRange()` reprend une convention de vérification déjà présente dans le projet.

Difficulté rencontrée : 9 au lieu de 3

Lors du premier contrôle, le résultat observé était 9 alors que 3 était attendu pour E4 → G4. Le problème venait de l'inversion des arguments dans l'appel, pas de la formule.

```
getIntervalSemitones(64, 67); // 3 : E -> G
getIntervalSemitones(67, 64); // 9 : G -> E
```

Cette erreur a confirmé une décision utile : **le premier argument représente la tonique de référence** et le second représente la note mesurée. L'ordre n'est pas interchangeable.

Une nuance de vocabulaire / API

Le nom `tonicMidi` a été conservé dans la fonction écrite pendant la séance, bien que l'interface lui passe également `tonic` (un entier de 0 à 11). Le calcul est correct car il opère modulo 12. Un renommage vers `tonic` ou `tonicPitchClass` serait plus explicite, mais **n'a pas été effectué dans cette séance**.

Détail JavaScript

L'opérateur `%` conserve le signe du dividende : `-9 % 12` vaut `-9`. On utilise donc la fonction `modulo()` du projet pour ramener ce résultat à 3.

SÉQUENCE 03

Associer demi-tons et libellés

Le choix retenu est un **tableau indexé par le nombre de demi-tons**. Un `Record` ou un `Map` serait possible, mais ce tableau est le plus direct : les indices vont précisément de 0 à 11.

```
export const INTERVALS = [
  "1", "b2", "2", "b3", "3", "4",
  "#4", "5", "b6", "6", "b7", "7"
] as const;
```

Erreur corrigée pendant la séance

La première proposition comptait **11 éléments** et sautait la tierce majeure `"3"`, normalement à l'indice 4. À partir de cette position, les libellés étaient décalés. Le tableau corrigé compte 12 éléments : l'indice devient une correspondance fiable pour chacun des 12 demi-tons.

Pourquoi as const ?

Avec `as const`, TypeScript infère un **tuple en lecture seule** dont les éléments conservent leurs valeurs littérales. Le tableau n'est pas simplement un `string[]`.

```
type IntervalLabel = (typeof INTERVALS)[number];
// "1" | "b2" | "2" | "b3" | "3" | ... | "7"

type Premier = (typeof INTERVALS)[0]; // "1"
type Quatrieme = (typeof INTERVALS)[3]; // "b3"
```

Ici, `[number]` est un **indexed access type** : il récupère le type de toutes les valeurs accessibles par un indice numérique. À l'inverse, `[3]` vise une position précise du tuple.

Transformer la distance en symbole

```
export function getIntervalLabel(
  tonic: number,
  noteMidi: number
): string {
  const semiTone = getIntervalSemitones(tonic, noteMidi);
  return INTERVALS[semiTone] ?? '1';
}
```

Point de vigilance non corrigé

Le repli `?? "1"` a été conservé dans le code écrit : si un index invalide apparaissait, il masquerait l'anomalie en présentant la note comme une tonique. Cette valeur de secours est superflue si le tableau et le modulo respectent leur contrat. Une stratégie explicite pourra être étudiée lors d'un futur durcissement, sans modifier rétroactivement la séance.

SÉQUENCE 04

Introduire un état d'affichage React

Les notes, la gamme et la sélection existaient déjà. Nous avons ajouté **un seul état de présentation** : faut-il afficher les noms des notes ou leurs degrés ?

```
const [displayMode, setDisplayMode] =
  useState<"notes" | "intervals">("notes");
```

L'union littérale interdit toute troisième valeur. Le choix initial `"notes"` garantit que l'interface démarre dans la présentation connue des séances précédentes.

Gérer le changement sans assertion de type

```
const handleDisplayModeChange = (
  { target }: React.ChangeEvent<HTMLSelectElement>
) => {
  const selectedMode = target.value;
  if (selectedMode === "notes" || selectedMode === "intervals") {
    setDisplayMode(selectedMode);
  }
};
```

Un élément HTML `select` fournit un `string`. Le test explicite réduit ce type à `"notes" | "intervals"` dans le bloc : c'est le **type narrowing**. Il n'est pas nécessaire d'écrire une assertion potentiellement trompeuse comme `as "notes" | "intervals"`.

Ajouter un select contrôlé

```
<div className="display-mode-option">
  <label className="option-label" htmlFor="mode-display">
    Affichage
  </label>
  <select
    name="display"
    id="mode-display"
    value={displayMode}
    onChange={handleDisplayModeChange}
  >
    <option value="notes">Notes</option>
    <option value="intervals">Intervalles</option>
  </select>
</div>
```

L'attribut `htmlFor` relie le libellé au contrôle identifié par `id`. `value` fait de ce sélecteur un **composant contrôlé** ; `onChange` fait évoluer l'état React.

Mécanisme de rendu

Quand `displayMode` change, React relance le rendu du composant. Il n'est pas nécessaire de créer un `useEffect` pour dériver le texte des cases. Un nouveau rendu n'implique pas nécessairement une modification effective de tous les nœuds du DOM.

SÉQUENCE 05

Brancher le libellé sur le manche

Le composant parcourt les cases de chaque corde avec `fretNumbers.map(...)`. Il calcule déjà la hauteur MIDI, la sélection et l'appartenance à la gamme. Nous avons branché le texte sur ces données sans modifier la coloration ni les interactions.

```
fretNumbers.map((fret) => {
  const isSelected = cellIsSelected(
    string.pos, fret, selectedPosition
  );
  const midiNote = getMidiNote(string.openMidi, fret);
  const label = displayMode === "notes"
    ? getFullNoteName(midiNote)
    : getIntervalLabel(tonic, midiNote);
  const isInScale = isNoteInScale(midiNote, tonic, scaleName);
  const isTonic = midiNote % 12 === tonic % 12;
  const note = isSelected || isInScale ? label : "";

  return (
    <div
      key={fret}
      className={`fret-cell ${isSelected ? "is-selected" : ""}
        ${isInScale ? "in-scale" : ""}
        ${isTonic ? "is-tonic" : ""}`}
      onClick={() => handleFretClick(string.pos, fret)}
      data-note={note}
    >
      <div
        className="string-line"
        style={{ "--i": string.pos } as React.CSSProperties}
      />
    </div>
  );
})
```

Deux corrections utiles dans ce bloc

- **Choix du texte** : l'opérateur ternaire appelle `getFullNoteName()` en mode Notes et `getIntervalLabel()` en mode Intervalles.
- **Choix des positions visibles** : la première version écrivait `const note = isSelected ? label : ""`. Seule la case sélectionnée montrait un texte. La version finale utilise `isSelected || isInScale` pour afficher toutes les notes de gamme, ainsi qu'une note hors gamme sélectionnée.

Il faut déclarer `isInScale` **avant** `note`, car la nouvelle condition en dépend. Les classes CSS `is-selected`, `in-scale` et `is-tonic` restent identiques.

HTML / CSS existant

Le libellé est transmis dans l'attribut `data-note` de la case. Le style exact qui rend cet attribut visible appartient au CSS déjà présent ; aucune nouvelle règle CSS n'a été fournie ni validée pendant cette séance.

SÉQUENCE 06

Procédure de reconstruction

Cette section sert de **mode opératoire** pour refaire uniquement la fonctionnalité du chapitre 4 dans une base Modal Guitar Lab équivalente à la fin de la séance 3.

Ordre	Fichier / zone	Opération reproductible
1	src/music/scales.ts	Conserver modulo() et isNoteInScale() ; ajouter getIntervalSemitones().
2	src/music/scales.ts	Ajouter INTERVALS dans l'ordre strict de 0 à 11 demi-tons.
3	src/music/scales.ts	Ajouter getIntervalLabel(tonic, noteMidi).
4	Composant du manche / parent	Ajouter displayMode (union "notes" | "intervals") initialisé à "notes".
5	Même composant	Créer handleDisplayModeChange() avec garde de type.
6	JSX des sélecteurs	Ajouter le select contrôlé et son label.
7	Rendu des fret-cell	Calculer label, isInScale puis note ; transmettre note à data-note.
8	Navigateur	Vérifier les deux modes, les degrés, la sélection hors gamme et les couleurs.
9	Terminal	Exécuter pnpm build ; résultat restant à confirmer.

Contrats entre modules

- `getMidiNote(openMidi, fret)` doit renvoyer une hauteur MIDI cohérente pour chaque case.
- `getFullNoteName(midiNote)` fournit le libellé utilisé en mode Notes. Son format exact (octave incluse ou non) reste celui de l'application existante.
- `checkMidiRange(value)` doit être accessible à la logique des intervalles, selon l'organisation réelle du projet ; la séance n'a pas documenté son import.
- `modulo(n, 12)` doit retourner un entier compris entre 0 et 11, y compris pour `n < 0`.
- `isNoteInScale(midiNote, tonic, scaleName)` doit continuer à fonctionner indépendamment du mode d'affichage.

Invariants à préserver

La **tonique** reste la référence commune au calcul des couleurs et des intervalles. Le **mode musical** contrôle l'appartenance à la gamme. Le **mode d'affichage** ne change que le libellé. La **sélection** reste stockée indépendamment : changer « Notes » en « Intervalles » n'altère pas la position choisie.

> **Portabilité du chapitre**
>
> Les noms de composants, les imports complets, le CSS de `data-note` et le démarrage Vite/pnpm n'ont pas été fournis dans cette séance. Pour une reconstruction intégrale depuis zéro, les chapitres 1 à 3 doivent documenter ces bases. Le présent chapitre documente intégralement le delta fonctionnel de la séance 4.

SÉQUENCE 07

Tests, incidents et validation

Tests de logique proposés pendant la séance

```
getIntervalLabel(4, 67); // "b3" : E -> G
getIntervalLabel(4, 71); // "5"  : E -> B
getIntervalLabel(4, 73); // "6"  : E -> C#
getIntervalLabel(4, 76); // "1"  : E -> E
```

Les quatre résultats correspondent au contrat attendu. Ces lignes ont été proposées comme vérification ; aucun banc de tests automatisés ni sortie complète d'exécution n'a été fourni dans le chat.

Vérifications fonctionnelles réellement confirmées

Scénario navigateur	Attendu	Statut déclaré
Lancer Mi dorien + Intervalles	1, 2, b3, 4, 5, 6, b7	OK
Sélectionner C naturel hors gamme	Affiche b6 sur la case choisie	OK
Revenir en mode Notes	Les noms réapparaissent	OK
Conserver sélection et couleurs	Aucun changement visuel collatéral	OK
Basculer Notes / Intervalles	Select fonctionne dans les deux sens	OK

Registre des difficultés et solutions

Difficulté observée	Diagnostic / correction
Résultat 9 au lieu de 3	Arguments tonic et note inversés lors du test ; rétablir tonic en premier.
Table INTERVALS de 11 valeurs	Tierce majeure "3" oubliée ; réinsérer à l'indice 4.
target.value typé string	Garde if sur les deux valeurs admises : TypeScript affine le type.
Affichage sur une seule case	Remplacer isSelected seul par isSelected || isInScale.
Ordre des déclarations	Calculer isInScale avant la variable note.
Libellé de secours potentiellement faux	Le ?? "1" a été discuté mais conservé ; risque identifié.

Contrôle encore ouvert

La compilation et le build final `pnpm build` ont été demandés en fin de séance, mais leur résultat n'apparaît pas dans la conversation. Le document ne les présente donc pas comme validés.

Décisions, acquis et reprise

Décisions techniques conservées

Choix	Motif / conséquence
Un tableau INTERVALS ordonné	Association directe entre indice 0–11 et libellé ; pas de Map ni de Record nécessaire.
as const	Retrouver des types littéraux fiables et dériver IntervalLabel sans duplication.
Une fonction musicale dédiée	Garder le calcul indépendant du composant et réutilisable.
Un état displayMode seulement	Ne pas stocker dans React des libellés qui se déduisent déjà de tonic et noteMidi.
Un select contrôlé	L'interface reflète la seule source de vérité displayMode.
Aucun changement de couleur	Le mode d'affichage ne modifie pas le modèle musical ni les repères de sélection.

Notions travaillées

- **Musique** : demi-tons, classe de hauteur, octave, degré, enharmonie et caractéristique du dorien.
- **JavaScript** : sens de la soustraction, opérateur modulo et gestion des valeurs négatives, opérateur `||`, condition ternaire, repli `??`.
- **TypeScript** : tuples `as const`, accès indexé aux types, unions littérales, garde de type et événement `ChangeEvent`.
- **React** : état contrôlé, rerender, calcul pendant le rendu et séparation entre données musicales et présentation.

État de fin de séance

Acquis : le manche peut afficher les noms des notes ou les intervalles des positions appartenant à la gamme, ainsi que le libellé de la position sélectionnée. Les cinq vérifications manuelles ont été déclarées satisfaisantes.

Restant explicitement ouvert : confirmation de `pnpm build`, tests automatisés ciblés éventuels, traitement du repli `?? "1"`, amélioration future de la notation enharmonique.

Point de départ du chapitre 5

Une piste cohérente est de **mettre en évidence les degrés caractéristiques des modes** (par exemple 6 en dorien, b2 en phrygien), tout en gardant le même principe : un incrément court, une proposition de code écrite par l'utilisateur, une vérification et une documentation de ce qui a réellement changé.

Prompt de reprise condensé

« Reprendre Modal Guitar Lab après quatre séances. Le manche 6 cordes (0–12) dispose de tonic, scaleName et displayMode, des modes mineur naturel / majeur / dorien / phrygien, de getIntervalSemitones(), INTERVALS et getIntervalLabel(). L'affichage Notes / Intervalles et les couleurs sont fonctionnels. Commencer par vérifier le build s'il reste à faire, puis choisir un petit incrément d'exploration modale. Me laisser coder, vérifier chaque étape et documenter la séance. »

Fin du chapitre 04 • Document conçu pour accompagner une reconstruction progressive et explicable du projet.